ARQUITECTURA FUNCIONAL TO-BE DEL ERP GECO REINGENIERIZADO

1. VISIÓN GENERAL DE LA ARQUITECTURA

La arquitectura funcional TO-BE del ERP Geco reingenierizado representa una evolución radical respecto al sistema heredado, abandonando el esquema monolítico basado en PHP/CodeIgniter y Sencha ExtJS para adoptar un ecosistema moderno de microservicios completamente desacoplados. Esta nueva arquitectura se fundamenta en los principios de escalabilidad horizontal, mantenibilidad, seguridad por diseño y experiencia de usuario centrada en las personas.

La arquitectura se organiza en tres capas principales claramente diferenciadas: la capa de presentación (frontend), la capa de lógica de negocio (backend) y la capa de persistencia (base de datos). Cada una de estas capas se comunica mediante contratos bien definidos (APIs RESTful), garantizando el desacoplamiento y permitiendo la evolución independiente de cada componente.

El flujo de la información comienza en el frontend, que interactúa con el usuario y genera peticiones HTTP hacia el backend. Estas peticiones son recibidas por un API Gateway, que actúa como punto único de entrada, enrutando las solicitudes hacia los microservicios correspondientes según la URL y las reglas de negocio. Cada microservicio procesa la petición, aplica la lógica de negocio correspondiente, consulta o actualiza la base de datos según corresponda, y devuelve una respuesta estructurada al frontend.

La siguiente tabla resume los principios arquitectónicos que guían el diseño de la nueva solución:

Principio	Descripción	Beneficio
Desacoplamiento por dominios	Cada módulo funcional (Productos, Proveedores, Reservas, Personas, Seguridades) es un microservicio independiente.	Escalabilidad independiente, mantenibilidad y evolución autónoma.
Comunicación RESTful	Interacción mediante APIs HTTP con intercambio de datos en formato JSON.	Interoperabilidad, desacoplamiento y contratos explícitos.

Separación de responsabilidades	Capas bien definidas: presentación (Angular), lógica de negocio (Spring Boot), persistencia (MySQL).	Mantenibilidad, pruebas y evolución tecnológica.
Seguridad por diseño	Autenticación JWT, autorización granular, validación de entradas.	Protección de datos y control de acceso robusto.
Trazabilidad y auditoría	Registro de operaciones relevantes y cambios de estado.	Control administrativo y resolución de incidencias.
Diseño centrado en el usuario	Interfaces intuitivas, responsivas y validadas con usuarios reales.	Productividad, satisfacción y reducción de errores.

2. ARQUITECTURA DE CAPAS DEL BACKEND

El backend se organiza en una estructura de capas bien definida que sigue el patrón de diseño Arquitectura Hexagonal o Puertos y Adaptadores, adaptado al ecosistema Spring Boot. Esta organización garantiza una clara separación de responsabilidades y facilita el mantenimiento y las pruebas.

La siguiente tabla describe cada una de las capas del backend:

Capa	Componentes	Responsabilidad	Tecnología
Capa de Controladores (Controllers)	REST Controllers, DTOs	Recibir peticiones HTTP, validar datos de entrada, delegar ejecución a servicios, devolver respuestas estructuradas. No contiene lógica de negocio.	Spring Web, Jakarta Validation
Capa de Servicios (Services)	Service Interfaces, Service Implementations, Business Logic	Encapsular la lógica de negocio, reglas de validación complejas, orquestación de operaciones y gestión de transacciones.	Spring Service, @Transactional

Capa de Repositorios (Repositories)	JPA Repositories, Custom Queries	Gestionar el acceso a la base de datos, proporcionar métodos de consulta y persistencia. Abstraer la tecnología de persistencia subyacente.	Spring Data JPA, Hibernate
Capa de Entidades (Entities)	JPA Entities, Relationships	Representar el modelo de datos, mapear tablas de la base de datos, definir relaciones y restricciones de integridad.	JPA, Hibernate, MySQL
Capa de DTOs (Data Transfer Objects)	Request DTOs, Response DTOs	Desacoplar la estructura interna de las entidades de los contratos de API. Facilitar la evolución independiente de ambos.	Java Records, POJOs
Capa de Seguridad (Security)	JWT Filter, Authentication Manager, Authorization Rules	Gestionar autenticación y autorización, proteger endpoints, validar tokens JWT.	Spring Security, JWT

3. ESTRUCTURA DE CAPAS DEL FRONTEND

El frontend se construye sobre una arquitectura modular basada en componentes, siguiendo las mejores prácticas de Angular. Cada módulo funcional es un módulo Angular independiente que se carga de forma perezosa (lazy-loading), lo que optimiza el rendimiento de la aplicación.

Capa	Componentes	Responsabilidad	Tecnología
Módulos (Modules)	Feature Modules, Shared Modules	Agrupar componentes, servicios, rutas y modelos por dominio funcional. Carga perezosa para optimización.	Angular NgModule, Lazy Loading
Componentes (Components)	UI Components, Container Components	Renderizar la interfaz de usuario, manejar eventos del usuario, gestionar el estado local.	Angular Components, HTML, SCSS
Servicios (Services)	HTTP Services, State Management Services	Comunicarse con el backend, gestionar el estado de la	Angular Services, HttpClient, RxJS

		aplicación, manejar autenticación y autorización.	
Modelos (Models)	TypeScript Interfaces, Classes	Definir la estructura de los datos intercambiados con el backend. Corresponden a los DTOs.	TypeScript
Interceptores (Interceptors)	HTTP Interceptors	Añadir token JWT a las peticiones, manejar errores de forma centralizada, registrar logs.	Angular HTTP Interceptor
Guards (Guards)	Route Guards	Proteger rutas, redirigir según el estado de autenticación y los roles del usuario.	Angular CanActivate, CanLoad

4. MÓDULOS FUNCIONALES Y SUS RESPONSABILIDADES

El sistema se organiza en cinco módulos funcionales independientes, cada uno implementado como un microservicio en el backend y como un módulo Angular en el frontend. Cada módulo encapsula su dominio de negocio, su lógica y sus datos, minimizando las dependencias entre ellos.

4.1. Módulo de Seguridades

Aspecto	Descripción
Responsabilidad	Gestión de autenticación, autorización, usuarios, perfiles, permisos y objetos del sistema.
Funcionalidades principales	Autenticación JWT, CRUD de usuarios, CRUD de perfiles, asignación de permisos, gestión de objetos del sistema, auditoría de accesos.
Dependencias	Ninguna (módulo base). Es consumido por todos los demás módulos.
Tecnologías clave	Spring Security, JWT, BCrypt.

4.2. Módulo de Personas

Aspecto	Descripción
Responsabilidad	Registro maestro de todos los actores del sistema: usuarios, clientes, proveedores.

Funcionalidades principales	CRUD de personas, gestión de identificaciones, direcciones, teléfonos, correos, parámetros (tipos de documento, nacionalidades, estados civiles).
Dependencias	Seguridades (autenticación y autorización).
Consumidores	Módulos de Proveedores y Reservas (sincronización de datos).

4.3. Módulo de Productos

Aspecto	Descripción
Responsabilidad	Parametrización de la oferta turística: productos, itinerarios, componentes, tarifas e impuestos.
Funcionalidades principales	CRUD de operaciones, UEN, temporadas, impuestos, tipos de producto, tipos de comida; creación de productos; gestión de itinerarios; gestión de componentes; gestión de tarifas; consultas y reportes.
Dependencias	Seguridades (autenticación y autorización), Personas (proveedores asociados a componentes).
Consumidores	Módulo de Reservas (consume productos, itinerarios, tarifas).

4.4. Módulo de Proveedores

Aspecto	Descripción
Responsabilidad	Gestión de proveedores, insumos, listas de precios y relaciones comerciales.
Funcionalidades principales	CRUD de proveedores, gestión de identificaciones, direcciones, teléfonos, relaciones, configuraciones financieras, insumos, listas de precios (básicas y avanzadas), búsquedas y reportes.
Dependencias	Seguridades (autenticación y autorización), Personas (sincronización de datos de proveedores).
Consumidores	Módulo de Reservas (consume insumos y proveedores).

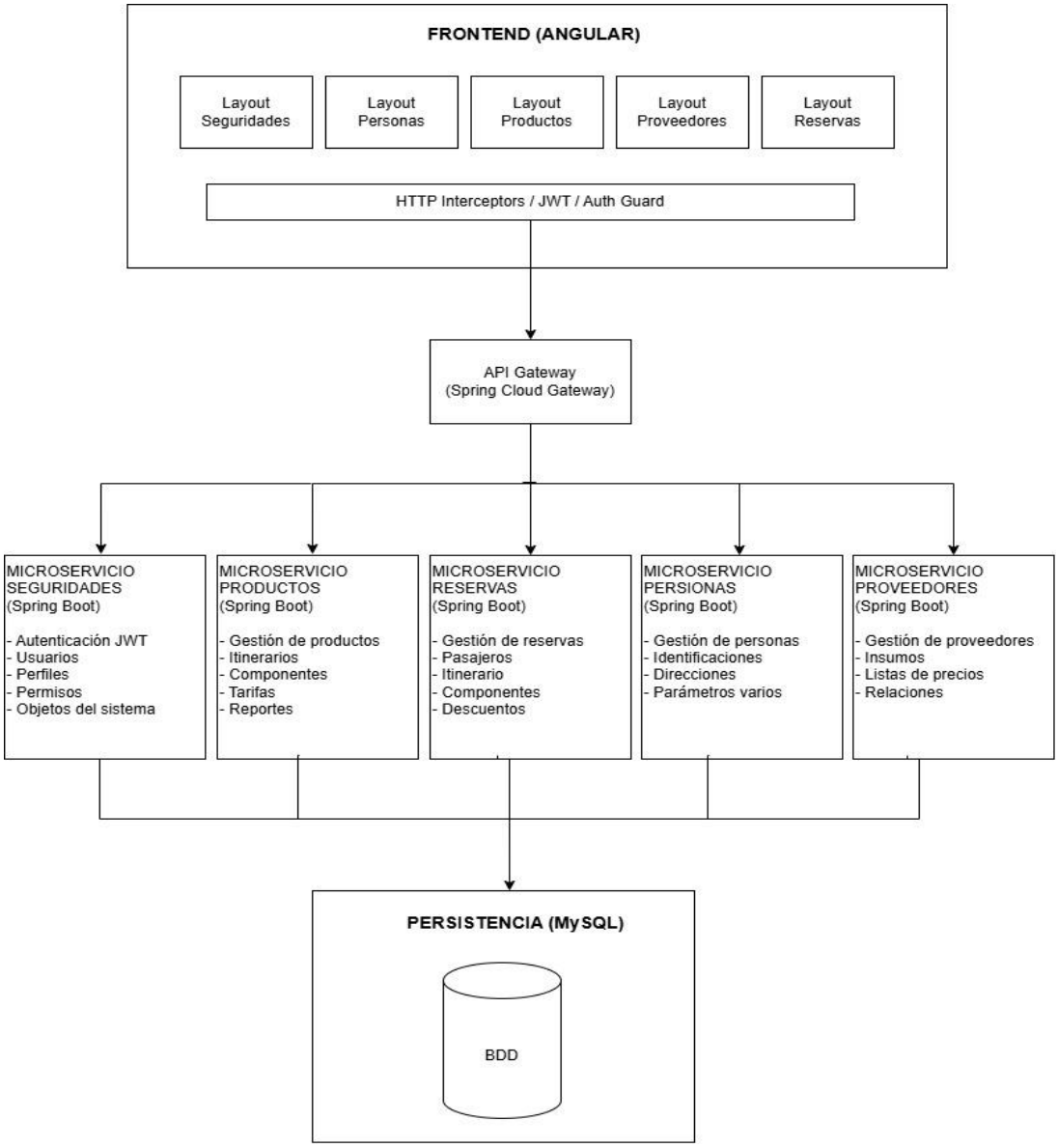
4.5. Módulo de Reservas

Aspecto	Descripción

Responsabilidad	Núcleo operativo del sistema: gestión integral de reservas, pasajeros, itinerarios, insumos, descuentos, anticipos y documentos.
Funcionalidades principales	CRUD de reservas, gestión de pasajeros, rooming list, inserción de productos con procesos automáticos, planificación de itinerarios, gestión de componentes e insumos, asignación de insumos por pasajero, descuentos y anticipos, cambio de estado, documentos operativos y reportes financieros.
Dependencias	Seguridades (autenticación y autorización), Personas (clientes), Productos (productos, itinerarios, tarifas), Proveedores (insumos, proveedores).
Consumidores	Ninguno (módulo final).

5. DIAGRAMA DE COMPONENTES Y COMUNICACIÓN

5.1. Vista lógica de la arquitectura



5.2. Patrón de comunicación entre módulos

La comunicación entre los módulos se realiza siguiendo un patrón de orquestación síncrona mediante APIs RESTful. Cada módulo expone sus servicios a través de endpoints que pueden ser consumidos por otros módulos cuando sea necesario.

Origen	Destino	Propósito	Tipo de comunicación
Reservas	Productos	Obtener itinerarios, insumos y tarifas de un producto al insertarlo en la reserva.	Síncrona (HTTP REST)
Reservas	Proveedores	Obtener información de insumos y proveedores para asignar servicios en la reserva.	Síncrona (HTTP REST)
Reservas	Personas	Obtener o sincronizar datos de clientes asociados a la reserva.	Síncrona (HTTP REST)
Proveedores	Personas	Sincronizar datos de proveedores al crearlos o actualizarlos.	Síncrona (HTTP REST)
Productos	Proveedores	Obtener datos de insumos y proveedores al configurar componentes.	Síncrona (HTTP REST)
Todos	Seguridades	Autenticar usuarios y verificar permisos en cada operación.	Síncrona (HTTP REST)

6. TECNOLOGÍAS Y HERRAMIENTAS

Capa	Tecnología	Versión	Propósito
Frontend	Angular	21+	Desarrollo de la interfaz de usuario, componentes reutilizables y gestión del estado.
Frontend	TypeScript	5.x	Lenguaje tipado para el desarrollo frontend.
Frontend	RxJS	7.x	Programación reactiva y manejo de flujos de datos asíncronos.
Backend	Java	21 LTS	Lenguaje de programación del backend.
Backend	Spring Boot	3.x	Framework para desarrollo de microservicios RESTful.

Backend	Spring Security	6.x	Gestión de autenticación y autorización.
Backend	Spring Data JPA	3.x	Abstracción de persistencia y mapeo objeto-relacional.
Backend	Hibernate	6.x	Implementación de JPA para ORM.
Persistencia	MySQL	8.0	Sistema de gestión de bases de datos relacional.
Seguridad	JWT (JSON Web Tokens)	-	Tokens de autenticación stateless.
Comunicación	RESTful APIs	-	Intercambio de datos entre frontend y backend.
Comunicación	JSON	-	Formato de intercambio de datos.
Documentación	Swagger / OpenAPI	3.x	Documentación interactiva de la API.
Control de versiones	Git / GitHub	-	Gestión del código fuente y colaboración.
Gestión de proyectos	Monday	-	Planificación de sprints y seguimiento de tareas.
Diagramas	Draw.io	-	Elaboración de diagramas UML y wireframes.

7. ESTRATEGIA DE DESPLIEGUE

La estrategia de despliegue propuesta para el ERP Geco reingenierizado se basa en los principios de contenedorización y orquestación, permitiendo un despliegue ágil, escalable y reproducible.

7.1. Estrategia de contenerización

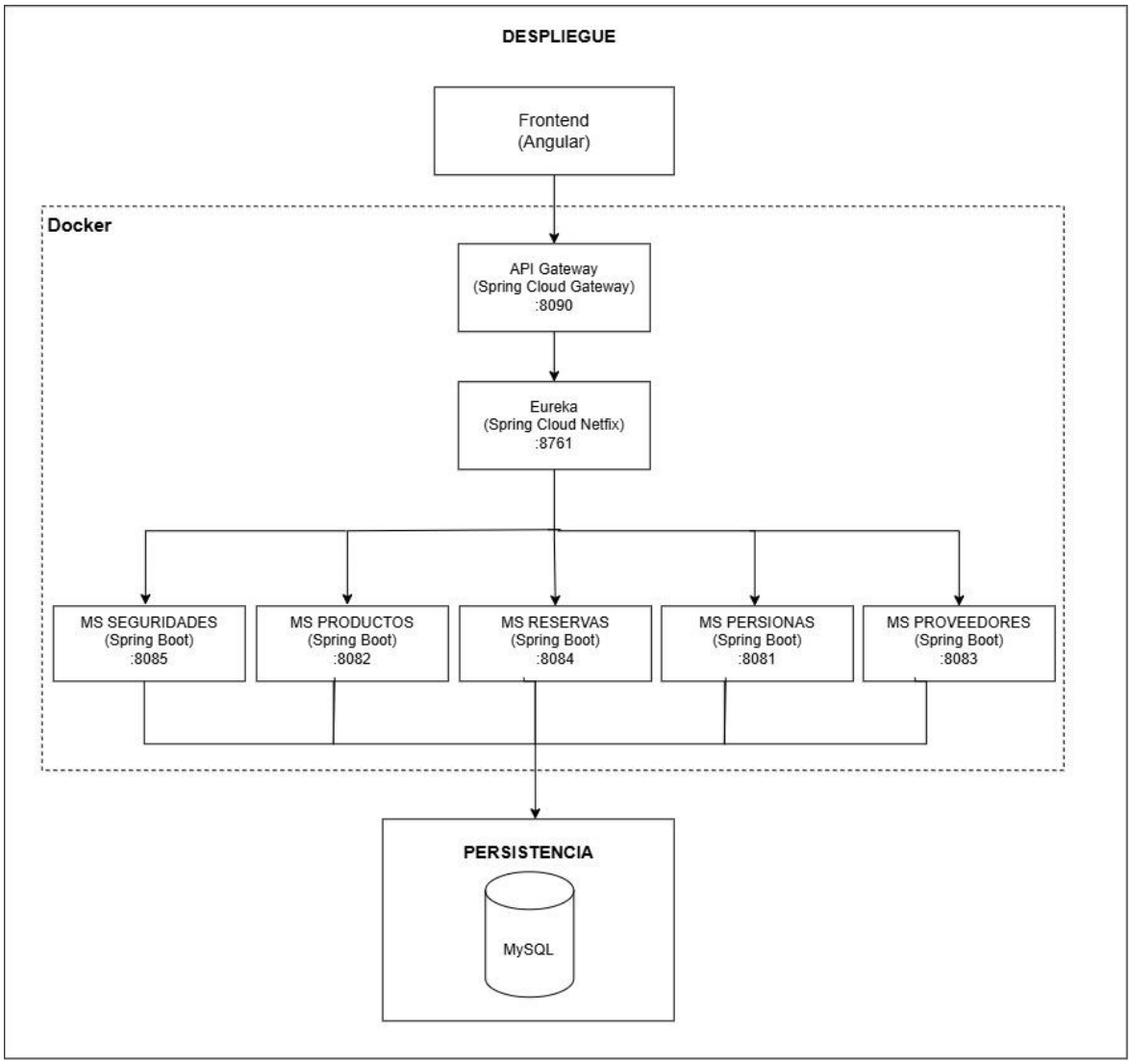
Cada microservicio se empaqueta como un contenedor Docker independiente, lo que garantiza la consistencia entre entornos y facilita el escalado horizontal. La siguiente tabla detalla los componentes a contenerizar:

Componente	Tecnología	Puerto	Observaciones
API Gateway	Spring Cloud Gateway	8090	Punto único de entrada
Eureka	Spring Boot	8761	
MS Seguridades	Spring Boot	8085	Autenticación y gestión de usuarios

MS Personas	Spring Boot	8081	Gestión de personas
MS Productos	Spring Boot	8082	Gestión de productos
MS Proveedores	Spring Boot	8083	Gestión de proveedores
MS Reservas	Spring Boot	8084	Gestión de reservas

Componente	Tecnología	Observaciones
Frontend	Angular	Se despliega compilada en el servidor web.
Base de datos	MySQL	Es independiente, no se conteneriza.

7.2. Arquitectura de despliegue



7.3. Entornos de despliegue

Entorno	Propósito	Configuración
Desarrollo (Local)	Desarrollo y pruebas unitarias	Cada desarrollador ejecuta los contenedores localmente con Docker Compose
Pruebas (Testing)	Pruebas funcionales y de integración	Entorno aislado en servidor de pruebas
Pre-producción (Staging)	Validación final con datos reales	Entorno similar a producción
Producción (Production)	Sistema en operación real	Alta disponibilidad, monitorización y backup

8. ESTRATEGIA DE SEGURIDAD

La seguridad se integra en todos los niveles de la arquitectura, siguiendo el principio de seguridad por diseño. La siguiente tabla detalla los mecanismos implementados en cada capa:

Capa	Mecanismo	Descripción
Frontend	Interceptores HTTP	Añaden el token JWT a las cabeceras de todas las peticiones
Frontend	Route Guards	Protegen las rutas según el estado de autenticación y roles del usuario
Frontend	Validación de formularios	Validan los datos antes de enviarlos al backend
Backend	Spring Security	Filtro de autenticación JWT, autorización por roles y permisos
Backend	Validación de entradas	Jakarta Bean Validation para prevenir inyecciones y datos inválidos
Backend	JWT	Tokens firmados con expiración controlada y refresh token
Backend	BCrypt	Almacenamiento seguro de contraseñas

Backend	CORS Configuration	Política de Cross-Origin Resource Sharing configurada para permitir solo el frontend
Backend	Consultas parametrizadas	Uso de JPA/Hibernate para prevenir SQL Injection
Persistencia	Encriptación de datos sensibles	(Propuesto) Encriptación de datos personales y financieros
Persistencia	Backup y recuperación	(Propuesto) Estrategia de backup y recuperación ante fallos

9. ESTRATEGIA DE TRAZABILIDAD Y AUDITORÍA

La trazabilidad se implementa de forma transversal en todos los módulos, garantizando que las operaciones relevantes queden registradas y sean auditables.

Aspecto	Mecanismo	Descripción
Auditoría de datos	Auditoría automática con Spring Data JPA	Anotaciones @CreatedBy, @CreatedDate, @LastModifiedBy, @LastModifiedDate
Auditoría de accesos	Logging de intentos de acceso	Registro de intentos de inicio de sesión (exitosos y fallidos)
Auditoría de cambios	Histórico de modificaciones	Registro de usuario, fecha, tipo de cambio y valores previos
Auditoría de estado	Histórico de cambios de estado	Registro de transiciones de estado en reservas y proveedores
Auditoría de acciones críticas	Logging específico	Registro de acciones como cambios de precio, descuentos, eliminaciones

10. ESTRATEGIA DE RENDIMIENTO Y ESCALABILIDAD

Aspecto	Estrategia	Descripción
Escalabilidad horizontal	Microservicios independientes	Permite escalar cada microservicio según su demanda específica

Escalado automático	Kubernetes HPA (propuesto)	Escalado automático basado en métricas de CPU y memoria
Caché	Redis / Caffeine (propuesto)	Caché de consultas frecuentes y datos maestros
Optimización de consultas	Índices JPA y SQL	Índices en campos de búsqueda y foráneos
Paginación	Spring Data Pageable	Paginación en todos los listados para evitar sobrecarga
Procesos asíncronos	Async / @Async (propuesto)	Procesos pesados (generación de documentos, cálculos) en segundo plano
Carga perezosa	Angular Lazy Loading	Carga de módulos solo cuando se necesitan

11. ESTRATEGIA DE PRUEBAS

Tipo de prueba	Herramientas	Cobertura objetivo
Pruebas unitarias (backend)	JUnit 5, Mockito	>70% de cobertura
Pruebas unitarias (frontend)	Jasmine, Karma	>70% de cobertura
Pruebas de integración	Spring Boot Test, TestContainers	Endpoints y servicios críticos
Pruebas funcionales	Postman / Newman	Todas las funcionalidades
Pruebas de usabilidad	Usuarios reales	Flujos completos de los tres módulos
Pruebas de rendimiento	K6 (propuesto)	Endpoints críticos

12. RELACIÓN DE ENDPOINTS POR MÓDULO

12.1. Módulo de Seguridades

- POST /api/v1/auth/login – Autenticación
- POST /api/v1/auth/logout – Cierre de sesión

- POST /api/v1/auth/refresh – Renovación de token
- GET /api/v1/usuarios – Listado de usuarios
- GET /api/v1/usuarios/{id} – Obtener usuario
- POST /api/v1/usuarios – Crear usuario
- PUT /api/v1/usuarios/{id} – Actualizar usuario
- DELETE /api/v1/usuarios/{id} – Eliminar usuario (lógico)
- GET /api/v1/perfiles – Listado de perfiles
- GET /api/v1/perfiles/{id} – Obtener perfil
- POST /api/v1/perfiles – Crear perfil
- PUT /api/v1/perfiles/{id} – Actualizar perfil
- GET /api/v1/permisos/objetos – Obtener objetos del sistema
- POST /api/v1/permisos/asignar – Asignar permisos por perfil
- GET /api/v1/permisos/usuario/{usuarioId} – Obtener permisos de usuario
- GET /api/v1/auditoria/accesos – Obtener registro de accesos

12.2. Módulo de Personas

- GET /api/v1/personas – Listado de personas
- GET /api/v1/personas/{id} – Obtener persona
- POST /api/v1/personas – Crear persona
- PUT /api/v1/personas/{id} – Actualizar persona
- DELETE /api/v1/personas/{id} – Eliminar persona (lógico)
- GET /api/v1/personas/buscar – Búsqueda avanzada de personas
- GET /api/v1/personas/{id}/identificaciones – Obtener identificaciones
- POST /api/v1/personas/{id}/identificaciones – Añadir identificación
- GET /api/v1/personas/{id}/direcciones – Obtener direcciones
- POST /api/v1/personas/{id}/direcciones – Añadir dirección
- GET /api/v1/personas/{id}/telefonos – Obtener teléfonos
- POST /api/v1/personas/{id}/telefonos – Añadir teléfono
- GET /api/v1/parametros/tipos-documento – Listado de tipos de documento

- GET /api/v1/parametros/nacionalidades – Listado de nacionalidades
- GET /api/v1/parametros/estados-civiles – Listado de estados civiles

12.3. Módulo de Productos

- GET /api/v1/productos – Listado de productos
- GET /api/v1/productos/{id} – Obtener producto
- POST /api/v1/productos – Crear producto
- PUT /api/v1/productos/{id} – Actualizar producto
- PATCH /api/v1/productos/{id}/estado – Cambiar estado del producto
- DELETE /api/v1/productos/{id} – Eliminar producto (lógico)
- GET /api/v1/productos/buscar – Búsqueda avanzada de productos
- GET /api/v1/productos/{id}/itinerario – Obtener itinerario del producto
- POST /api/v1/productos/{id}/itinerario/dias – Añadir día al itinerario
- PUT /api/v1/productos/{id}/itinerario/dias/{diaId} – Actualizar día
- DELETE /api/v1/productos/{id}/itinerario/dias/{diaId} – Eliminar día
- GET /api/v1/productos/{id}/itinerario/dias/{diaId}/componentes – Obtener componentes del día
- POST /api/v1/productos/{id}/itinerario/dias/{diaId}/componentes – Añadir componente
- PUT /api/v1/productos/{id}/itinerario/componentes/{compId} – Actualizar componente
- DELETE /api/v1/productos/{id}/itinerario/componentes/{compId} – Eliminar componente
- GET /api/v1/productos/{id}/tarifas – Obtener tarifas del producto
- POST /api/v1/productos/{id}/tarifas – Añadir tarifa
- PUT /api/v1/productos/{id}/tarifas/{tarifaId} – Actualizar tarifa
- DELETE /api/v1/productos/{id}/tarifas/{tarifaId} – Eliminar tarifa
- GET /api/v1/operaciones – Listado de operaciones
- GET /api/v1/uen – Listado de UEN
- GET /api/v1/temporadas – Listado de temporadas
- GET /api/v1/impuestos – Listado de impuestos
- GET /api/v1/tipos-producto – Listado de tipos de producto

- GET /api/v1/tipos-comida – Listado de tipos de comida
- GET /api/v1/productos/reportes/itinerario/{productoId} – Reporte de itinerario
- GET /api/v1/productos/reportes/precios/{productoId} – Reporte de precios

12.4. Módulo de Proveedores

- GET /api/v1/proveedores – Listado de proveedores
- GET /api/v1/proveedores/{id} – Obtener proveedor
- POST /api/v1/proveedores – Crear proveedor
- PUT /api/v1/proveedores/{id} – Actualizar proveedor
- PATCH /api/v1/proveedores/{id}/estado – Cambiar estado del proveedor
- DELETE /api/v1/proveedores/{id} – Eliminar proveedor (lógico)
- GET /api/v1/proveedores/buscar – Búsqueda avanzada de proveedores
- GET /api/v1/proveedores/{id}/identificaciones – Obtener identificaciones
- POST /api/v1/proveedores/{id}/identificaciones – Añadir identificación
- GET /api/v1/proveedores/{id}/direcciones – Obtener direcciones
- POST /api/v1/proveedores/{id}/direcciones – Añadir dirección
- GET /api/v1/proveedores/{id}/telefonos – Obtener teléfonos
- POST /api/v1/proveedores/{id}/telefonos – Añadir teléfono
- GET /api/v1/proveedores/{id}/relaciones – Obtener relaciones
- POST /api/v1/proveedores/{id}/relaciones – Añadir relación
- GET /api/v1/proveedores/{id}/insumos – Obtener insumos del proveedor
- POST /api/v1/proveedores/{id}/insumos – Añadir insumo
- PUT /api/v1/insumos/{id} – Actualizar insumo
- DELETE /api/v1/insumos/{id} – Eliminar insumo
- GET /api/v1/insumos/buscar – Búsqueda de insumos
- GET /api/v1/insumos/{id}/precios – Obtener precios del insumo
- POST /api/v1/insumos/{id}/precios – Añadir precio
- PUT /api/v1/insumos/{id}/precios/{precioId} – Actualizar precio

- DELETE /api/v1/insumos/{id}/precios/{precioId} – Eliminar precio
- GET /api/v1/tipos-insumo – Listado de tipos de insumo
- GET /api/v1/clasificadores – Listado de clasificadores
- GET /api/v1/unidades-medida – Listado de unidades de medida
- GET /api/v1/proveedores/reportes/listado – Reporte de listado de proveedores

12.5. Módulo de Reservas

- GET /api/v1/reservas – Listado de reservas
- GET /api/v1/reservas/{id} – Obtener reserva
- POST /api/v1/reservas – Crear reserva
- PUT /api/v1/reservas/{id} – Actualizar reserva
- PATCH /api/v1/reservas/{id}/estado – Cambiar estado
- POST /api/v1/reservas/copiar – Copiar reserva/proforma
- DELETE /api/v1/reservas/{id} – Anular reserva
- GET /api/v1/reservas/{id}/pasajeros – Obtener pasajeros
- POST /api/v1/reservas/{id}/pasajeros – Añadir pasajero
- POST /api/v1/reservas/{id}/pasajeros/masivo – Carga masiva de pasajeros
- PUT /api/v1/reservas/{id}/pasajeros/{paxId} – Actualizar pasajero
- DELETE /api/v1/reservas/{id}/pasajeros/{paxId} – Eliminar pasajero
- GET /api/v1/reservas/{id}/rooming – Obtener rooming list
- POST /api/v1/reservas/{id}/rooming – Crear rooming list
- PUT /api/v1/reservas/{id}/rooming/{roomingId} – Actualizar rooming
- DELETE /api/v1/reservas/{id}/rooming/{roomingId} – Eliminar rooming
- GET /api/v1/reservas/{id}/detalle – Obtener detalle de productos
- POST /api/v1/reservas/{id}/detalle – Añadir producto al detalle
- PUT /api/v1/reservas/{id}/detalle/{detalleId} – Actualizar detalle
- DELETE /api/v1/reservas/{id}/detalle/{detalleId} – Eliminar detalle
- GET /api/v1/reservas/{id}/itinerario – Obtener itinerario de la reserva

- PUT /api/v1/reservas/{id}/itinerario/dias/{diaId} – Actualizar día
- POST /api/v1/reservas/{id}/itinerario/dias – Añadir día
- POST /api/v1/reservas/{id}/itinerario/recalcular-fechas – Recalcular fechas
- GET /api/v1/reservas/{id}/itinerario/dias/{diaId}/componentes – Obtener componentes
- POST /api/v1/reservas/{id}/itinerario/dias/{diaId}/componentes – Añadir componente
- PUT /api/v1/reservas/{id}/itinerario/componentes/{compId} – Actualizar componente
- DELETE /api/v1/reservas/{id}/itinerario/componentes/{compId} – Eliminar componente
- POST /api/v1/reservas/{id}/componentes/asignar-pax – Asignar insumo por pasajero
- POST /api/v1/reservas/{id}/descuentos – Aplicar descuento global
- POST /api/v1/reservas/{id}/detalle/{detalleId}/descuentos – Aplicar descuento por detalle
- POST /api/v1/reservas/{id}/anticipos/cliente – Registrar anticipo de cliente
- POST /api/v1/reservas/{id}/anticipos/proveedor – Registrar anticipo a proveedor
- GET /api/v1/reservas/{id}/resumen-financiero – Obtener resumen financiero
- GET /api/v1/reservas/{id}/documentos/itinerario – Generar itinerario
- GET /api/v1/reservas/{id}/documentos/voucher/{paxId} – Generar voucher por pasajero
- GET /api/v1/reservas/{id}/documentos/rooming – Generar rooming list
- GET /api/v1/reservas/{id}/documentos/orden-servicios – Generar orden de servicios
- GET /api/v1/reservas/{id}/documentos/confirmacion-proveedor – Generar confirmación
- GET /api/v1/reservas/{id}/documentos/cierre-file – Generar cierre de file
- GET /api/v1/reservas/{id}/documentos/pagos-cliente – Pagos por cliente
- GET /api/v1/reservas/{id}/documentos/pagos-proveedor – Pagos a proveedores
- GET /api/v1/reservas/{id}/documentos/gastos-tour – Gastos por tour
- GET /api/v1/parametros/motivos – Listado de motivos diversos
- GET /api/v1/parametros/tipos-reserva – Listado de tipos de reserva

13. ESTRATEGIA DE INTEGRACIÓN CONTINUA (CI/CD)

La estrategia de integración continua y despliegue continúo propuesta para el proyecto se basa en los siguientes principios:

Fase	Herramientas	Descripción
Compilación	Maven (backend), Angular CLI (frontend)	Compilación y empaquetado de los módulos
Pruebas	JUnit, Jasmine, Karma, Newman	Ejecución de pruebas unitarias y funcionales automatizadas
Análisis de código	SonarQube	Análisis estático de código, detección de vulnerabilidades y deuda técnica
Empaquetado	Docker	Creación de imágenes Docker de cada microservicio
Monitorización	Herramientas Spring Boot	Monitorización de métricas, logs y alertas